INVEST SAFE LUDO

Local 4-Player Board Game — Full Build Guide for Cursor / any AI IDE

This document is a complete, section-wise build specification for the **NSE Invest Safe Ludo** game. Paste each section (or the whole document) into Cursor, Windsurf, Copilot Chat, or any AI coding tool, and it has everything needed to build the app end-to-end: exact board layout, colours, game rules, scam/protection square effects, player-data capture, and Excel export — with no hosting or backend server required.

Players	4 (fixed) — local, same laptop, pass-and-play, turn based
Hosting	None. Runs 100% locally in the browser — just double-click and open the HTML file, or run one local dev command.
Core loop	Standard Ludo movement + 20 special squares (10 Scam, 10 Protection) with unique effects
Player data	Name, Organisation, Phone number — captured before play starts
Data export	One click → downloads a real .xlsx (Excel) file to the laptop

Section 1 — Recommended Tech Stack

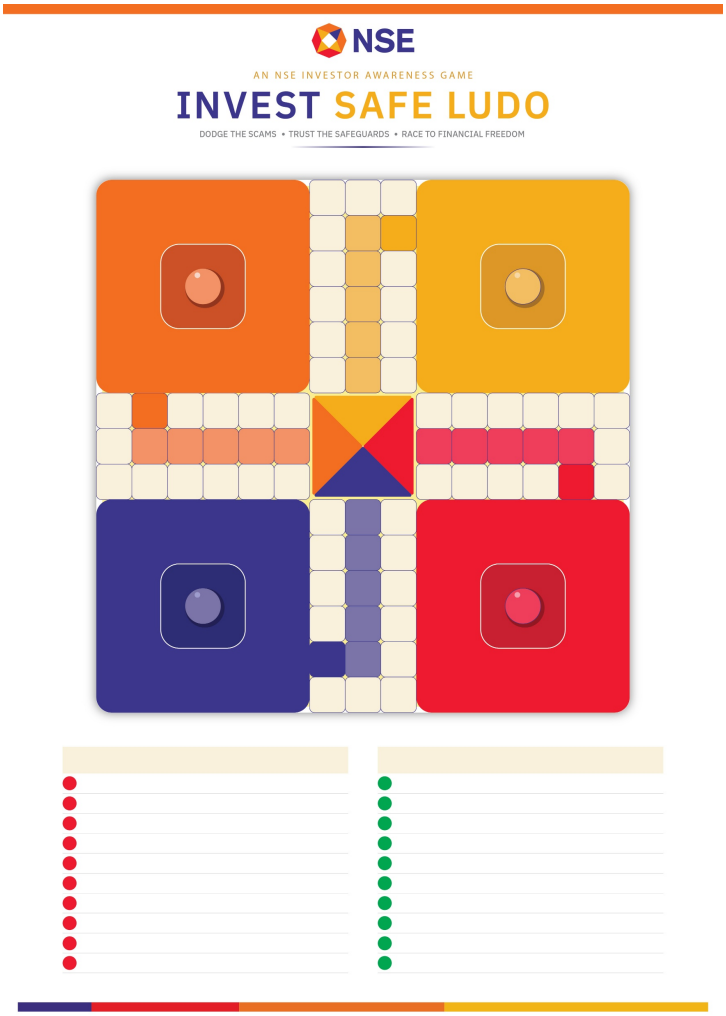
Because the game only needs to run locally with no hosting, the simplest and most bug-resistant option is a **single-page React app** that runs entirely in the browser — no backend, no database server, no deployment.

Layer	Choice	Why
UI framework	React (Vite)	Fast local dev server, component-based, easy state management for 4 players/turns
Board rendering	SVG / CSS Grid	Pixel-precise board matching the reference design, scales cleanly, no image-slicing bugs
Dice	CSS 3D-transform animated cube (or a small canvas dice)	Looks like a real rolling die, no physics engine needed = no glitches
State management	React useState / useReducer	4 players, positions, turn order — small enough to not need Redux
Data storage	In-memory state, mirrored to localStorage	Survives accidental refresh; no server/database needed
Excel export	SheetJS (xlsx npm package)	Generates a real .xlsx file and downloads it — pure client-side, no backend
Packaging	Vite build → static HTML/CSS/JS folder	Just open index.html or run one local server command — nothing to host online

Why not a backend (FastAPI/Node) even though it's local? Because nothing needs to be shared over a network — everything happens on one laptop, in one browser tab. Adding a server just adds more things that can break (port conflicts, CORS, process crashing) for zero benefit here. If a backend is ever wanted later (e.g. syncing results across many laptops at an event), FastAPI + SQLite can be added on top of this exact same frontend without rewriting the game logic.

Section 2 — Visual Reference: Keep This Exact Board

The board layout below is from the original NSE deck. Cursor must reproduce this exact look — same 4-colour corner bases, same cream path squares, same centre triangle 'home' — not a generic Ludo board.



Reference board exported directly from NSE_INVEST_SAFE_LUDO.pptx — use as the visual source of truth.

Element	Approx. HEX	Swatch
Orange base (top-left / Player 1)	#F36E21	
Gold/Yellow base (top-right / Player 2)	#F5A623	
Navy Blue base (bottom-left / Player 3)	#2E2B6B	
Red base (bottom-right / Player 4)	#E12038	
Cream path squares (background)	#F9F1DC	
Pink-red path column accent	#EF3E5B	
Purple-blue path column accent	#7B74A8	

Section 3 — Board Grid & Special Square Positions

The board is a standard 15×15 Ludo grid (row 0 and col 0 = top-left corner). Below are the exact grid positions of all 20 special squares, measured directly from the source design — use these so the special squares land in the same visual spots as the reference board.

Scam Squares (10) — red/negative

ID	Row,Col	Theme	Effect on landing
S1	12,7	Fake Trading App	Send token back to Home (start)
S2	7,3	Investment Tips (unverified)	Send token back to Home (start)
S3	3,8	Opinion Trading Scam	Skip next turn
S4	8,12	Pump and Dump Scam	Send token back to Home (start)
S5	11,8	Fake Trading Course	Move back 5 steps
S6	8,4	Deepfake Video	Skip next 2 turns
S7	4,7	Digital Arrest Call	Move back 3 steps
S8	7,11	Foreign Portfolio Scam	Skip next 2 turns
S9	7,1	Dabba Trading	Skip the next Protection square's bonus
S10	7,14	Guaranteed Returns	Move back 2 steps

Protection Squares (10) — green/positive

ID	Row,Col	Theme	Effect on landing
P1	12,8	SEBI Verified App	Move ahead 2 steps
P2	11,7	Spot-a-Scam Tool	Roll the dice once more
P3	8,3	SCORES / ODR	Move ahead 5 steps
P4	5,7	SEBI Registered Broker	Roll the dice once more
P5	1,7	Visit a CISC	Move ahead 3 steps
P6	14,7	Report Cyber Fraud	Immune to the next Scam square hit
P7	7,4	Verify before Trust	Move ahead 2 steps
P8	7,12	SEBI CHECK	Roll the dice once more
P9	8,11	Deal with Registered IA	Immune to the next Scam square hit
P10	5,9	Verify Rumours	Move ahead 2 steps

Note: (Row,Col) is on a 0-indexed 15×15 grid, top-left = (0,0). These squares sit on the main outer path (never inside a home base or the coloured home-stretch column).

Section 4 — Core Game Rules

- 4 fixed players: Orange, Yellow (Gold), Navy Blue, Red — matching the 4 home corners.
- Each player has 1 token (keep it simple — 1 token per player, not the classic 4-tokens-each, unless you want the harder version later).
- Players take turns in a fixed order (Orange → Yellow → Blue → Red → repeat).
- On a turn: click/tap the dice → animated roll → shows a number 1-6 → token moves that many squares along the path.
- Rolling a 6 grants one extra roll immediately after (standard Ludo rule) — keep this, it keeps the game fast for 4 players.
- If a token lands exactly on a Scam square (S1-S10), apply that square's effect immediately (see Section 3 table).
- If a token lands exactly on a Protection square (P1-P10), apply that square's effect immediately.
- If a player is currently 'Immune' (from P6/P9) and lands on a Scam square, the scam effect is skipped once and immunity is consumed.
- First token to complete the full path and reach the centre triangle Home wins the game.
- After a winner is decided, show a results screen with final standings and the 'Export to Excel' button.

Section 5 — Player Registration & Excel Export

Before the board screen appears, show a registration screen collecting exactly 3 fields per player, for all 4 players:

Field	Type	Notes
Name	Text, required	Free text
Organisation	Text, required	Company / institution name
Phone Number	Text, required	10-digit validation recommended

Store this data in React state for the session, and also mirror it to **localStorage** so a session isn't lost if the tab refreshes. When the game ends (or via an 'Export' button visible any time), generate an .xlsx file using SheetJS with one row per player:

```
Columns: Player Colour | Name | Organisation | Phone Number | Final Position | Squares Landed On

import * as XLSX from 'xlsx';

function exportToExcel(players) {
  const rows = players.map(p => ({
    'Player': p.color,
    'Name': p.name,
    'Organisation': p.org,
    'Phone Number': p.phone,
    'Final Position': p.rank,
  }));
  const ws = XLSX.utils.json_to_sheet(rows);
  const wb = XLSX.utils.book_new();
  XLSX.utils.book_append_sheet(wb, ws, 'Players');
  XLSX.writeFile(wb, `Invest_Safe_Ludo_${Date.now()}.xlsx`);
}
```

Section 6 — Suggested Project Structure

```
invest-safe-ludo/
  index.html
  package.json
  src/
    main.jsx
    App.jsx
    components/
      RegistrationScreen.jsx
      Board.jsx
      Dice.jsx
      Token.jsx
      SquareEffectPopup.jsx
      ResultsScreen.jsx
    data/
      boardLayout.js // 15x15 grid + path order + S/P square positions (Section 3)
      squareEffects.js // scam & protection effect definitions (Section 3 tables)
    utils/
      gameLogic.js // dice roll, movement, turn order, win check
      exportExcel.js // SheetJS export function (Section 5)
    styles/
      board.css
```

To run locally: **npm install** then **npm run dev** — opens on http://localhost:5173 in the browser, on the same laptop, no internet required after the first install. For a completely offline double-click version, run **npm run build** and open the generated dist/index.html directly.

Section 7 — Paste-Ready Prompts for Cursor (build in this order)

Give these to Cursor one at a time, in order. Each one is self-contained — paste the whole box as your message to Cursor's chat/composer.

CURSOR PROMPT 1 — Project Setup

```
Create a new Vite + React project called 'invest-safe-ludo'. Set up the folder structure exactly as below, with empty placeholder files/components for now:

invest-safe-ludo/src/{main.jsx, App.jsx, components/{RegistrationScreen.jsx, Board.jsx, Dice.jsx, Token.jsx, SquareEffectPopup.jsx, ResultsScreen.jsx}, data/{boardLayout.js, squareEffects.js}, utils/{gameLogic.js, exportExcel.js}, styles/board.css}

Install dependencies: react, react-dom, xlsx (SheetJS). No backend, no server, no database — this is a fully client-side app that will only ever run on localhost for local play, never deployed online.

App.jsx should manage 3 screens via simple state: 'registration' -> 'game' -> 'results'.
```

CURSOR PROMPT 2 — Player Registration Screen

```
Build RegistrationScreen.jsx: a form for exactly 4 players (Orange, Yellow, Navy Blue, Red — show each player's colour as a swatch/label). For each player collect: Name (text, required), Organisation (text, required), Phone Number (text, required, validate 10 digits). On submit, save all 4 players' data into a players array in App.jsx state (fields: color, name, org, phone, position: 0, rank: null) and also write it to localStorage under key 'investSafeLudoPlayers'. Then move to the 'game' screen. Style it clean and branded: navy blue (#2E2B6B) and orange (#F36E21) as primary colours, white background, rounded input fields.
```

CURSOR PROMPT 3 — Board Layout & Rendering

```
Build boardLayout.js: define a 15x15 Ludo board as a standard cross-shaped path of 52 outer cells plus 6-cell coloured home-stretches for each of the 4 colours (Orange, Yellow, Navy Blue, Red), matching a classic Ludo layout. Each of the 4 home bases occupies a 6x6 corner block. The centre 3x3 is the winning triangle home, split into 4 coloured triangles matching the 4 players.

Then in the same file, place these 20 special squares at these exact (row, col) grid positions (0-indexed, row 0 = top):

SCAM squares: S1(12,7) S2(7,3) S3(3,8) S4(8,12) S5(11,8) S6(8,4) S7(4,7) S8(7,11) S9(7,1) S10(7,14)
PROTECTION squares: P1(12,8) P2(11,7) P3(8,3) P4(5,7) P5(1,7) P6(14,7) P7(7,4) P8(7,12) P9(8,11) P10(5,9)

Build Board.jsx to render this as CSS Grid (15x15), with:
- Orange home base top-left, Gold/Yellow home base top-right, Navy Blue home base bottom-left, Red home base bottom-right (each a large rounded-corner coloured square with a circular token holder icon in the middle)
- Cream (#F9F1DC) coloured plain path squares
- Scam squares shown as small red rounded badges labelled S1-S10
- Protection squares shown as small green rounded badges labelled P1-P10
- Centre 3x3 as 4 coloured triangles meeting at the middle
- 'INVEST SAFE LUDO' header banner above the board, tagline 'DODGE THE SCAMS • TRUST THE SAFEGUARDS • RACE TO FINANCIAL FREEDOM'

Reference image for the exact visual style is attached separately — match its look closely (colours, rounded corners, spacing).
```

CURSOR PROMPT 4 — Dice, Movement & Scam/Protection Logic

Build squareEffects.js exporting an object keyed by square ID with 'type' (scam/protection) and 'effect'. Use exactly this data:

SCAM: S1 Fake Trading App -> sendHome | S2 Investment Tips -> sendHome | S3 Opinion Trading Scam -> skipTurn(1) | S4 Pump and Dump Scam -> sendHome | S5 Fake Trading Course -> moveBack(5) | S6 Deepfake Video -> skipTurn(2) | S7 Digital Arrest Call -> moveBack(3) | S8 Foreign Portfolio Scam -> skipTurn(2) | S9 Dabba Trading -> skipNextProtectionBonus | S10 Guaranteed Returns -> moveBack(2)

PROTECTION: P1 SEBI Verified App -> moveAhead(2) | P2 Spot-a-Scam Tool -> rollAgain | P3 SCORES/ODR -> moveAhead(5) | P4 SEBI Registered Broker -> rollAgain | P5 Visit a CISC -> moveAhead(3) | P6 Report Cyber Fraud -> immuneNextScam | P7 Verify before Trust -> moveAhead(2) | P8 SEBI CHECK -> rollAgain | P9 Deal with Registered IA -> immuneNextScam | P10 Verify Rumours -> moveAhead(2)

Build Dice.jsx: a clickable die showing faces 1-6 using CSS 3D transforms (rotateX/rotateY) so it visually tumbles for about 700ms before landing on the rolled number. Disable the dice while a token is mid-move or it's not the current player's turn.

Build gameLogic.js: turn order cycling through the 4 players, dice roll (random 1-6), moving the current player's token that many steps along the path from boardLayout.js, applying squareEffects.js when a token lands exactly on a special square (show a short popup via SquareEffectPopup.jsx naming the square and its effect), rolling a 6 = extra turn, and detecting when a token completes the path (winner).

CURSOR PROMPT 5 — Results Screen & Excel Export

Build ResultsScreen.jsx: shown once a player wins. Display final standings (winner first) using each player's saved name/org/phone plus their colour. Add an 'Export to Excel' button.

Build exportExcel.js using the 'xlsx' package: one row per player with columns Player (colour), Name, Organisation, Phone Number, Final Position. Trigger a real .xlsx file download named Invest_Safe_Ludo_<timestamp>.xlsx when the button is clicked — this must work fully offline, no server call.

Also add a small 'Export to Excel' button visible during the game itself (not just at the end), in case someone wants the player list before the game finishes.

CURSOR PROMPT 6 — Final Polish & Testing

Polish pass: 1) Make sure the whole app works with npm run dev and also as a static build (npm run build) that can be opened locally without internet. 2) Add simple sound-free visual feedback: a subtle highlight animation on the token that just moved, and a coloured glow on a square when its effect triggers (red glow for scam, green glow for protection). 3) Make sure 4 players' turn order is always visible (e.g. a strip showing whose turn it is with their colour highlighted). 4) Test end-to-end: register 4 players -> play a full game -> reach a winner -> export Excel -> confirm the .xlsx opens correctly in Excel with the right columns and data.

Section 8 — Final Checklist Before Calling It Done

- 4 players can register with Name, Organisation, Phone before the board appears
- Board visually matches the reference image (colours, corner bases, cream path, centre triangle)
- All 20 special squares (S1-S10, P1-P10) are in the correct grid positions and trigger the correct effect
- Dice animates smoothly and never gets 'stuck' or clickable mid-roll
- Turn order cycles correctly through all 4 players, with a clear 'whose turn' indicator
- Rolling a 6 grants an extra turn
- Immunity from P6/P9 correctly cancels the next scam hit exactly once
- Game correctly detects a winner and shows the results screen
- Excel export produces a real, opens-correctly .xlsx file with all 4 players' data
- Entire app runs with zero internet connection after the first 'npm install'